COMP2007: Operating Systems & Concurrency
Week 11 – 12:00pm Wednesday – 04 December 2024

valid for 75 minutes from 11:45am
generated 2024-09-20 14:34

Figure: Attendance Monitoring

Operating Systems and Concurrency

File Systems 5

COMP2007

Geert De Maere
(Dan Marsden)
{Geert.DeMaere,Dan.Marsden}@Nottingham.ac.uk

University Of Nottingham
United Kingdom

2024

Recap

- File systems implementations
  - Contiguous
  - Linked lists
  - FAT
  - i-nodes
- File systems paradigms (on top of implementations) .
  - Log-structured file systems (improves efficiency)
  - Journaling (improves resiliency, robustness)
  - Virtual File Systems (improves flexibility, integration)

Goals for Today

Overview

- File system recovery
  - Scandisk
  - FSCK
- Links (hard and soft)
- Free space management and defragmentation
- File systems in Linux
- Example exercises

File System Consistency

Checking Consistency

- **Journaling reduces** the probability of inconsistencies, but does not completely avoid them
- This is problematic for **structural blocks** (e.g. **i-nodes**, **directories**, and **free lists**)
- **System utilities** utilities help to restore file systems, e.g.:
  - Scandisk
  - FSCK
- They check for **block** and **directory** consistency

File System Consistency

Checking Block Consistency

- Block consistency checks whether blocks are **assigned**/**used** the correct way
- Checked by building **two tables**, counting how often a block is present in:
  - **A file** (based on the i-nodes)
  - The **free list**
- A consistent file system has a **1 in either of the tables** for each block
- This is a **slow process** (hence why journaling is used as well)

File System Consistency

Checking Block Consistency

Block number	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
	1	1	0	1	0	1	1	1	1	0	0	1	1	1	0	0	Blocks in use
	0	0	1	0	1	0	0	0	0	1	1	0	0	0	1	1	Free blocks

Figure: Consistency checks (based on Tanenbaum)

File System Consistency

Checking Block Consistency

Block number	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
Blocks in use	1	1	0	1	0	1	1	1	1	0	0	1	1	1	0	0	
Free blocks	0	0	0	0	1	0	0	0	0	1	1	0	0	0	1	1	

Figure: Consistency checks (based on Tanenbaum)

File System Consistency

Checking Block Consistency

Block number	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
Blocks in use	1	1	0	1	0	1	1	1	1	0	0	1	1	1	0	0	
Free blocks	0	0	1	0	2	0	0	0	0	1	1	0	0	0	1	1	

Figure: Consistency checks (based on Tanenbaum)

File System Consistency

Checking Block Consistency

Block number	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
Blocks in use	1	1	0	1	0	2	1	1	1	0	0	1	1	1	0	0	
Free blocks	0	0	1	0	1	0	0	0	0	1	1	0	0	0	1	1	

Figure: Consistency checks (based on Tanenbaum)

©University of Nottingham

File System Consistency

Restoring Block Consistency

1. A **missing block**: it does not exist in any of the tables ⇒ add it to the free list
2. A **double counted block**:
   - It exists in both tables
   - Solution: re-build the free list (“disaster” waiting to happen otherwise)
3. A block is present in **two or more files**:
   - **Removing one file** results in adding the block to the **free list**
   - **Removing both files** will result in a **double entry** in the free list
   - **Solution:** make a **deep copy** of the block (one file is still likely to be damaged)

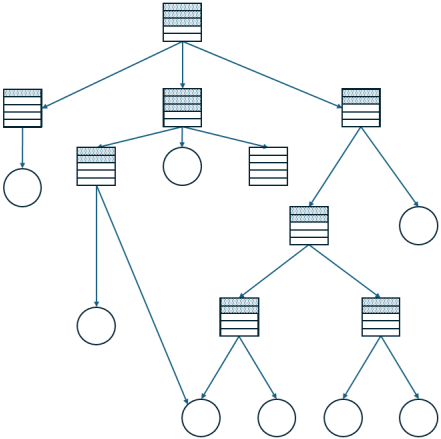
File System Consistency

Restoring Block Consistency

```
FSCK Algorithm:
 1. Iterate through all the i-nodes
    - retrieve the blocks
    - increment the counters
 2. Iterate through the free list
    - increment counters for free blocks
 3. Compare and resolve
```

i-nodes: Sharing files between directories

Hard and Soft Links



i-nodes: Sharing files between directories

Hard and Soft Links

- There are two approaches to **share a file**, e.g. between directory B and C, where C is the 'real' owner:
  - **Hard links**: maintain two (or multiple) references to the same i-node in B and C
    - the i-node link reference counter will be set to 2
  - **Symbolic links**:
    - The owner maintains a reference to the i-node in, e.g., directory C
    - The "referencer" maintains a small file (that has its own i-node) that contains the location and name of the shared file in directory C
- What is the **best approach**? $\Rightarrow$ both have advantages and disadvantages

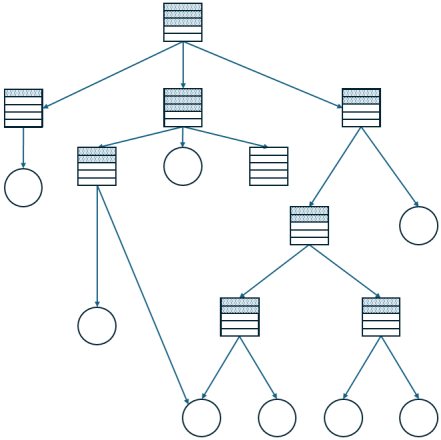
i-nodes: Sharing files between directories

Hard Links

- Hard links are the **fastest way of linking files**!
- Disadvantages of hard links:
  - Assume that the owner of the file **deletes** it:
    - If the i-node is also deleted, any hard link will, in the best case, **point to an invalid i-node**
    - If the i-node gets **deleted** and "**recycled**" to point to an other file, the hard links will **point to the wrong file**!
  - The only solution is to **delete the file**, and **leave the i-node intact** if the "**reference count**" is larger than 0 (the original owner of the file still gets "charged" for the space)

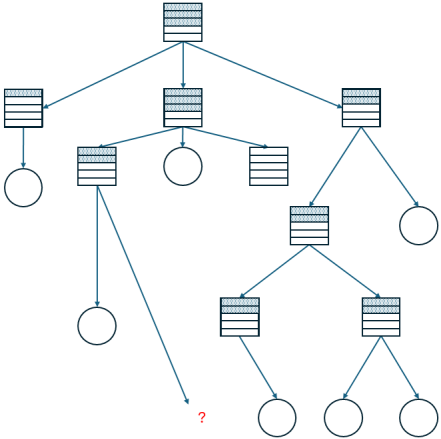
i-nodes: Sharing files between directories

Deleting Files



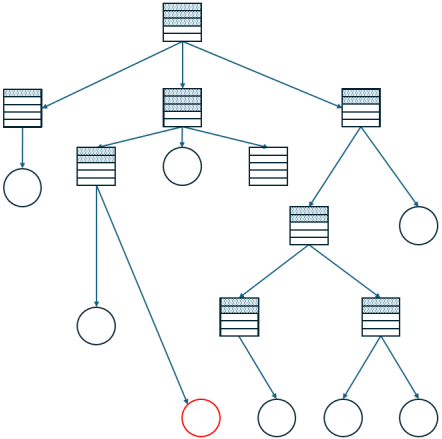
i-nodes: Sharing files between directories

Deleting Files



i-nodes: Sharing files between directories

Deleting Files



i-nodes: Sharing files between directories

Hard Links

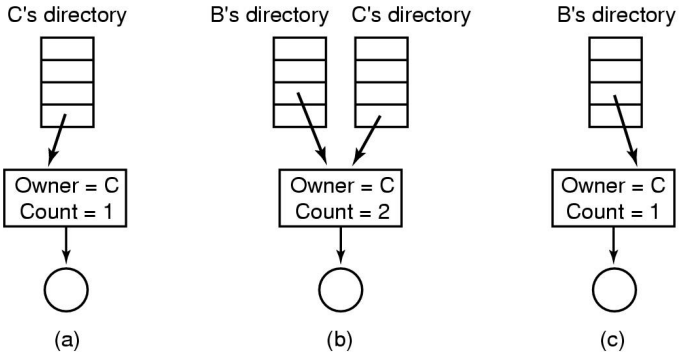


Figure: (a) Before creating a link. (b) After creating a link. (c) after the orignal owner removes the file (Tanenbaum)

i-nodes

Soft Links

- Advantages of symbolic links:
  - There are **no problems with deleting** the original file $\Rightarrow$ the file simply does not exist any more
  - They can **cross the boundaries of machines**, i.e. the linked file can be located on a different machine
- Disadvantages of soft links:
  - They result in an **extra file lookup** (once the link file has been found, the original file needs to be found as well)
  - They require an **extra i-node** for the link file

File System Consistency

Restoring i-node Consistency

- Checking the directory system: are the **i-node reference counters** correct?
- Possible problems:
  - **i-node counter is higher** than the number of directories containing the file
    - Removing the file will reduce the i-node counter by 1
    - Since the counter will remain larger than 1, the **i-node** / **disk space will not be released** for future use
  - **i-node counter is less** than the number of directories containing the file
    - Removing $N - 1$ files will set the **i-node counter to 0** whilst it is **still referenced** elsewhere

File System Consistency

Restoring i-node Consistency

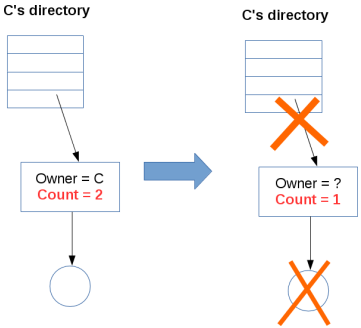


Figure: i-node counter is higher than the actual number of directories containing the file

File System Consistency

Restoring i-node Consistency

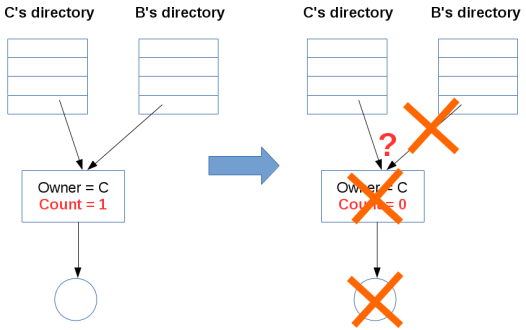


Figure: i-node counter is less than the actual number of directories containing the file

File System Consistency

Restoring i-node Consistency

```
- Recurse through the directory hierarchy
  - Increment file specific counters
  - I.e. each file is associated with one counter
- One file may appear in multiple directories
  - Compare the file counters and i-node counters
  - Correct if necessary
```

Free Space Management

Bitmaps

- Similar to memory management, **bitmaps** and **linked lists** can be used for free spa
management
- **Bitmaps** represent each block by a single bit in a map
  - The **size of the bitmap** grows with the size of the disk but is constant for a given disk
  - Bitmaps take **comparably less space than linked lists**

Figure: Disk Layout

Free Space Management

Linked Lists

- **Linked list** of free groupings / blocks
  - Use **free blocks** to hold the **locations of the free blocks** (hence, they are no longer free)
  - The size of the list **grows with the size of the disk** and **shrinks with the size of the blocks**
    - E.g., with a 1KB block a 32-bit/4 byte disk block number, each block will hold 255 free blocks (one for the pointer to the next block)
    - Since the free list shrinks when the disk becomes full, this is not wasted space
  - **Blocks are linked together**, i.e., multiple blocks list the free blocks
- Linked lists can be modified by **keeping track of the number of consecutive free blocks** for each entry (known as counting)

Free Space Management

Comparison

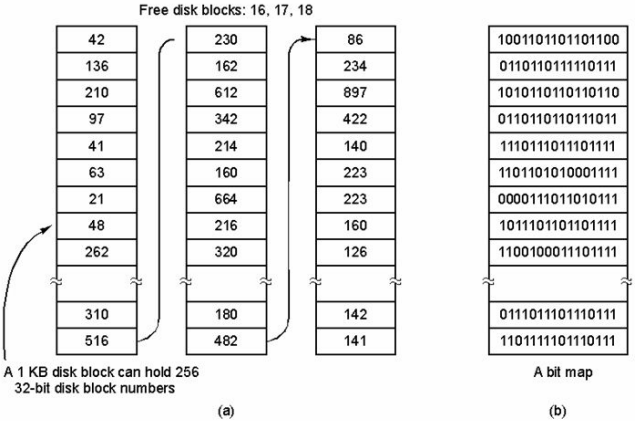


Figure: Free Block Management with Linked Lists (Tanenbaum)

Free Space Management

Bitmap vs. linked list

- Bitmaps:
  - Require extra space. E.g.: If block size = 2^{12} bytes (4KB) and disk size = 2^{30} bytes (1 GB) $\Rightarrow$ bitmap size: $2^{30}/2^{12} = 2^{18}$ (32KB)
  - Keeping it in main memory is possible only for small disks
- Linked lists:
  - **Grows** with the number of empty blocks
  - **No waste** of disk space (uses empty space)
  - We only need to keep in memory **one block of pointers** (load a new block when need)

©University of Nottingham

File System Defragmentation

Compacting

- At the beginning, all free disk space is in a single contiguous unit
- Removing and creating files results in fragmentation over time (blocks distributed across the disk)
  - **Defrag** utilities make file blocks contiguous (**very slow operation**, usually runs in the background)
  - They combine free space in one or more **large contiguous regions** (rotational hard drives)
- Linux (ext2/3/4) suffers less from fragmentation (block groups)

File System

History of the Linux file system

- **Minix file system**: the maximum file size was 64MB and file names were limited to 14 characters
- The "**extended file system**" (extfs): file names were 255 characters and the maximum file size was 2 GB
- The "**ext2**" file system: larger files, larger file names, better performance
- The "**ext3-4**" file system: journaling

File System

Standard Unix file system vs. Extended 2 File System

Figure: Standard Unix Partition

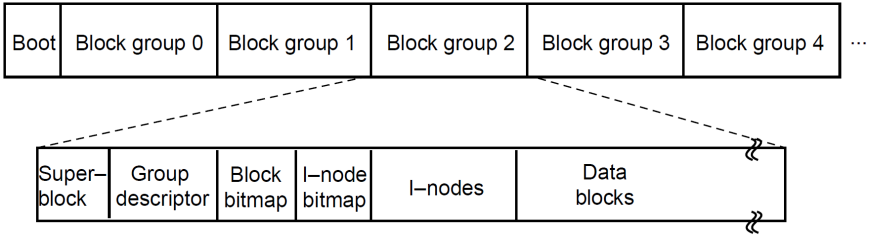


Figure: ext2 Partition Layout (Tanenbaum)

File System

Directory Entries

- The **superblock** contains file system information (e.g. the number of i-nodes, disk blocks)
- The **group descriptor** contains bitmap locations, the number of free blocks, i-nodes and directories
- A **data block bitmap** and **i-node bitmap**, used to keep track of free disk blocks and i-nodes (Unix uses lists)
- A **table of i-nodes** containing file and disk block information
- **Data blocks** containing file and directory blocks

©University of Nottingham

File System

The Extended 2 File System

- An ext2 partition is split into several **block groups** to:
  - **Reduce fragmentation** by storing i-nodes and files, and parent directories and files in the same block group if possible
  - Reduce **seek times** and improve performance
- All block groups have the same size and are stored sequentially (which allows direct indexing)

File System

Directory Entries

- Every **directory entry** contains the following **fixed-length** fields:
  - i-node number
  - Entry size in bytes
  - Type field, i.e. file, directory, special file, etc.
  - File name length in bytes
- And then, the file name itself (of **variable-length**)
- Directories are **searched linearly** (i.e. they are unsorted) and a cache is maintained for recently accessed items

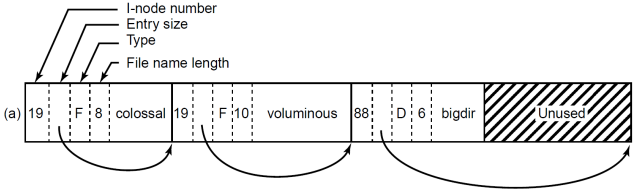


Figure: ext2 Directory Implementation (Tanenbaum)

File System

Directory Entries

- File names up to 255 characters
- **File lookups** are similar to the Unix file system
- The **i-node structure** is similar to the Unix i-nodes
  - 12 block addresses are contained in the i-node
  - Single, double and triple indirect blocks are used
  - With a blocks of 1KB, this scheme can handle file sizes of 16GB
  - If block size is 8KB, it could support file sizes up to 64TB

©University of Nottingham

File System

The Ext3 File System

- When making changes to an ext2 file system, files are ideally **written immediately** to prevent inconsistency:
  - This generates **significant head movement**
  - ext2 File system is more suitable for **flash disks** (no journal)
- Ext3 builds upon the ext2 file system by adding:
  - **Tree based structures** for directory files to facilitate indexing (HTrees)
  - **Journaling** capabilities

Test Your Understanding

File Systems

Extended 2 File System

And the **ext2** file system uses **12 direct block pointers** in the i-node and up to **triple indirect blocks**. Assuming that the block size is 4 kilobytes and 32 bit disk addresses are used:

- Could it store a file of 18GB?
- How many **disk blocks** are used for the i-node and indirect blocks for a 16MB file?

Test Your Understanding

Could We Store a File of 18GB?

- Pointers per block:

Test Your Understanding

Could We Store a File of 18GB?

- Pointers per block: we have blocks of 4KB and 32 bit addresses (4 bytes)
  - 1024 pointers per data block
  - 4KB = $4*2^{10}$ => $4*2^{10}$ / 4 bytes each = 2^{10} = 1024
- Total number of pointers:

Test Your Understanding

Could We Store a File of 18GB?

- Pointers per block: we have blocks of 4KB and 32 bit addresses (4 bytes)
  - 1024 pointers per data block
  - 4KB = $4*2^{10}$ => $4*2^{10}$ / 4 bytes each = 2^{10} = 1024
- Total number of pointers:
  - 12 direct block pointers
  - Indirect block pointers:

Test Your Understanding

Could We Store a File of 18GB?

- Pointers per block: we have blocks of 4KB and 32 bit addresses (4 bytes)
  - 1024 pointers per data block
  - 4KB = $4*2^{10}$ => $4*2^{10}$ / 4 bytes each = 2^{10} = 1024
- Total number of pointers:
  - 12 direct block pointers
  - Indirect block pointers:
    - Single $\Rightarrow$ 1024 extra block pointers
    - Double $\Rightarrow$ 1024×1024 block pointers
    - Triple $\Rightarrow$ $1024 \times 1024 \times 1024$ block pointers (1073741824)
  - Total 1,074,791,436 pointers
- Total file size:

Test Your Understanding

Could We Store a File of 18GB?

- Pointers per block: we have blocks of 4KB and 32 bit addresses (4 bytes)
  - 1024 pointers per data block
  - 4KB = $4*2^{10}$ => $4*2^{10}$ / 4 bytes each = 2^{10} = 1024
- Total number of pointers:
  - 12 direct block pointers
  - Indirect block pointers:
    - Single $\Rightarrow$ 1024 extra block pointers
    - Double $\Rightarrow$ 1024×1024 block pointers
    - Triple $\Rightarrow$ $1024 \times 1024 \times 1024$ block pointers (1073741824)
  - Total 1,074,791,436 pointers
- Total file size: 1,074,791,436 $\times$ 4KBs (approx. 4TB)

Test Your Understanding

How Many Disk Blocks are used for the i-node of 16MB file?

- Pointers required:

Test Your Understanding

How Many Disk Blocks are used for the i-node of 16MB file?

- Pointers required: $\frac{16 \times 2^{20}}{4 \times 2^{10}} = 2^{12} = 4096$ pointers
- The i-node requires one block [**1 block**]
- A single indirect block can contain 1024 block pointers [**1 block**]
- A double indirect block can address 1024×1024 block pointers
  - 1 double indirect block at first level [**1 block**]
  - (4096 - 1024 - 12)/1024 = 2.9882 double indirect blocks at 2nd level [**3 blocks**]
- Total: 6 blocks of 4KB $\Rightarrow$ 24KB

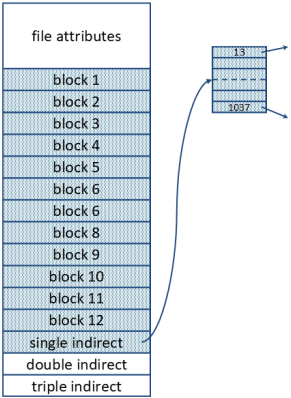


Figure: Types of access to a file

Test Your Understanding

How Many Disk Blocks are used for the i-node of 16MB file?

- Pointers required: $\frac{16 \times 2^{20}}{4 \times 2^{10}} = 2^{12} = 4096$ pointers
- The i-node requires one block [**1 block**]
- A single indirect block can contain 1024 block pointers [**1 block**]
- A double indirect block can address 1024×1024 block pointers
  - 1 double indirect block at first level [**1 block**]
  - (4096 - 1024 - 12)/1024 = 2.9882 double indirect blocks at 2nd level [**3 blocks**]
- Total: 6 blocks of 4KB $\Rightarrow$ 24KB

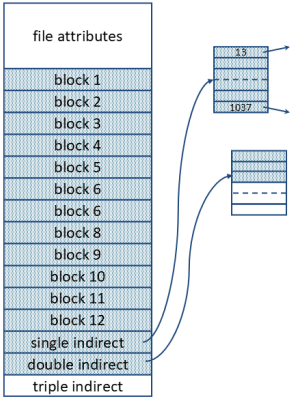


Figure: Types of access to a file

Test Your Understanding

How Many Disk Blocks are used for the i-node of 16MB file?

- Pointers required: $\frac{16 \times 2^{20}}{4 \times 2^{10}} = 2^{12} = 4096$ pointers
- The i-node requires one block [**1 block**]
- A single indirect block can contain 1024 block pointers [**1 block**]
- A double indirect block can address 1024×1024 block pointers
  - 1 double indirect block at first level [**1 block**]
  - (4096 - 1024 - 12)/1024 = 2.9882 double indirect blocks at 2nd level [**3 blocks**]
- Total: 6 blocks of 4KB $\Rightarrow$ 24KB

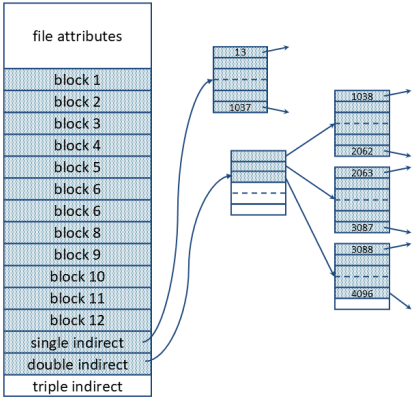


Figure: Types of access to a file

Test Your Understanding

File Lookups

Accessing Files

In Linux, how many look-ups are necessary to find (and load) the file: /opt/spark/bin/spark-shell?

Exercises

Lookups for retrieving /opt/spark/bin/spark-shell

```
- 1st: Find i-node for / (In Linux in a fixed position on the drive)
- 2nd: Find the actual file for / with its content
- 3rd: Find the i-node for /opt
- 4th: Find the actual file for /opt with its content
- 5th: Find the i-node associated to /opt/spark
- 6th: Find the actual file for /opt/spark/ with its content
- 7th: Find the i-node associated to /opt/spark/bin
- 8th: Find the actual file for /opt/spark/bin with its  content
- 9th: Find the i-node associated to /opt/spark/bin/spark-shell
- 10th: Find the desired file for /opt/spark/bin/spark-shell
```

©University of Nottingham

Summary

Take-Home Message

- File system consistency
- Free space management
- Linux file systems

©University of Nottingham